بخش اول

سلام و توضیحات اولیه جلسه

اسلاید اول

بلوک اول: کالبد فیزیکی اینترنت؛ کابل‌ها و شبکه‌ها

زمان هدف: ۱۵ دقیقه
اسلاید ۲

بخش اول: یک سؤال ساده؛ اینترنت کجاست؟ (۲ دقیقه)

«بچه‌ها، قبل از اینکه وارد بحث طراحی وب بشیم، می‌خوام با یک سؤال خیلی ساده شروع کنیم.

فرض کنید الان من ازتون بخوام گوشی‌هاتون رو بردارید و برید توی گوگل. مثلاً عبارت هواشناسی تهران رو جست‌وجو کنید.

چند ثانیه بعد، کلی اطلاعات، تصویر، نمودار و متن روی صفحه گوشی شما ظاهر می‌شه.

حالا سؤال من اینه: این اطلاعات دقیقاً از کجا اومدن؟

گوگل کجاست؟ اطلاعاتی که الان روی گوشی شما نمایش داده می‌شه، قبل از اینکه شما اون رو ببینید کجا بوده؟ چطوری از اونجا به گوشی شما رسیده؟

ما معمولاً می‌گیم اطلاعات رو از اینترنت گرفتیم. ولی اینترنت دقیقاً کجاست؟

آیا اینترنت یک نرم‌افزار خاصه؟ یک دستگاهه؟ یک ماهواره‌ست؟ آیا تمام اطلاعات دنیا توی یک کامپیوتر بزرگ نگهداری می‌شن؟

واقعیت اینه که اینترنت هیچ‌کدوم از این‌ها به‌تنهایی نیست.

برای اینکه بفهمیم اینترنت چیه، باید یک مقدار به عقب برگردیم و ببینیم اصلاً چرا کامپیوترها رو به هم وصل کردیم.»

برو اسلاید ۳

بخش دوم: از کامپیوترهای مستقل تا شبکه‌های کامپیوتری (۳ دقیقه)

«بیایید تصور کنیم چند دهه قبل زندگی می‌کنیم. کامپیوترها به شکل امروزی کوچک و قابل حمل نبودن. دستگاه‌های بزرگی بودن که معمولاً توی دانشگاه‌ها، مراکز تحقیقاتی و سازمان‌های بزرگ استفاده می‌شدن.

هر کامپیوتر برای خودش یک دنیای مستقل بود. اطلاعاتی که روی یک کامپیوتر وجود داشت، لزوماً روی کامپیوتر دیگه‌ای در دسترس نبود.

اگر یک محقق در یک دانشگاه اطلاعاتی تولید می‌کرد و محقق دیگری در دانشگاه دیگه‌ای به اون اطلاعات نیاز داشت، چطور باید این اطلاعات رو منتقل می‌کردن؟

احتمالاً اولین چیزی که به ذهنمون می‌رسه اینه که اطلاعات رو روی یک حافظه ذخیره کنیم و به دست طرف مقابل برسونیم. کاری که امروز هم با فلش مموری انجام می‌دیم.

اما این روش برای دنیایی که حجم اطلاعاتش دائماً در حال افزایش بود، روش مناسبی نبود. و یه نکته‌ی دیگه: اون موقع تجهیزات ذخیره‌سازی اطلاعات خیلی بزرگ و عملاً غیرقابل حمل بودن!

اینجا یک ایده مهم شکل گرفت: چرا کامپیوترها رو به هم متصل نکنیم تا بتونن مستقیماً با هم ارتباط برقرار کنن؟

این ایده منجر به شکل‌گیری چیزی شد که بهش می‌گیم شبکه کامپیوتری.

در ساده‌ترین حالت، شبکه یعنی چند دستگاه که به شکلی به هم متصل شدن و می‌تونن اطلاعات ردوبدل کنن.

مثلاً همین الان در خونه خودتون احتمالاً چند دستگاه به یک مودم متصل هستن؛ گوشی، لپ‌تاپ، تلویزیون هوشمند و شاید یک دستگاه دیگه.

این دستگاه‌ها یک شبکه کوچک تشکیل دادن.

شما می‌تونید از لپ‌تاپ به یک پرینتر متصل به شبکه دستور چاپ بدید، یا فایل‌هایی رو بین دستگاه‌های مختلف منتقل کنید.

پس برای داشتن یک شبکه، لزوماً به اینترنت نیاز نداریم. حتی اگر ارتباط اینترنت خونه‌تون قطع بشه، ممکنه همچنان دستگاه‌های داخل شبکه محلی بتونن با هم ارتباط داشته باشن.

به چنین شبکه‌ای LAN یا Local Area Network می‌گن.

حالا تصور کنید به‌جای چند کامپیوتر داخل یک خونه، بخوایم کامپیوترهای یک ساختمان، یک دانشگاه، یک شهر یا حتی چند کشور رو به هم متصل کنیم.

اینجاست که شبکه‌ها بزرگ‌تر می‌شن و شبکه‌های گسترده یا WAN شکل می‌گیرن یا Wide Area Network.

و این ما رو به داستان شکل‌گیری اینترنت می‌رسونه.»

برو اسلاید ۴

بخش سوم: تولد اینترنت؛ از ARPANET تا امروز (۳ دقیقه)

«در دهه ۱۹۶۰ میلادی، در آمریکا یک پروژه تحقیقاتی به نام ARPANET شکل گرفت.

این پروژه زیر نظر سازمانی به نام ARPA انجام می‌شد که در حوزه پروژه‌های تحقیقاتی پیشرفته فعالیت می‌کرد. و صد البته این سازمان هم زیر نظر وزارت دفاع آمریکا بود.

ایده اصلی این بود که کامپیوترهای مراکز تحقیقاتی مختلف بتونن به یکدیگر متصل بشن و اطلاعات و منابع محاسباتی رو به اشتراک بگذارند.

در سال ۱۹۶۹، اولین ارتباطات آزمایشی بین چند مرکز برقرار شد و ARPANET به‌تدریج توسعه پیدا کرد.

یک نکته جالب درباره اولین ارتباط معروف این شبکه وجود داره.

در یکی از آزمایش‌های اولیه، قرار بود کلمه LOGIN از یک دانشگاه به کامپیوتر دانشگاه دیگه ارسال بشه. اما سیستم بعد از دریافت دو حرف L و O از کار افتاد!

یعنی اولین پیام معروفی که قرار بود از طریق این شبکه منتقل بشه، عملاً فقط LO بود!

این داستان جالبه، چون نشون می‌ده چیزی که امروز ما به‌راحتی ازش استفاده می‌کنیم، چقدر مسیر طولانی و پر از آزمون و خطا داشته.

با گذشت زمان، شبکه‌های مختلفی در نقاط مختلف دنیا به وجود اومدن. شبکه‌های WAN مستقل از همدیگه. اما یک مشکل، یا بهتره بگم یه ایده وجود داشت: چطور می‌شد این شبکه‌های متفاوت رو به هم متصل کرد؟

برای حل این مشکل، مجموعه‌ای از استانداردهای ارتباطی توسعه پیدا کرد که مهم‌ترینشون TCP/IP بود.

در سال ۱۹۸۳، ARPANET به‌طور رسمی به استفاده از TCP/IP منتقل شد. این اتفاق یکی از نقاط عطف مهم در شکل‌گیری اینترنت مدرن محسوب می‌شه.

اینجا خیلی خلاصه بگم TCP/IP چیه؟
TCP/IP در واقع زبان مشترکیه که کامپیوترها با استفاده از اون می‌تونن از طریق شبکه با هم حرف بزنن. تصور کن می‌خوای برای دوستت نامه بفرستی؛ باید نامه رو داخل پاکت بذاری، آدرس دوستت رو روش بنویسی و به پست تحویل بدی. TCP/IP هم تقریباً همین کار رو می‌کنه: اطلاعات رو میگیره، آدرس مقصد رو مشخص می‌کنه و اون‌ها رو از مسیرهای مختلف به مقصد می‌رسونه. خلاصه اینکه TCP/IP مجموعه‌ای از قوانین و روش‌هاست که باعث می‌شه دستگاه‌های مختلف بتونن اطلاعات رو از طریق شبکه و اینترنت برای هم ارسال و دریافت کنن.

به‌مرور دانشگاه‌ها، مراکز تحقیقاتی، سازمان‌ها و بعدتر شرکت‌ها و کاربران عادی هم به این شبکه عظیم متصل شدن.
و چیزی که امروز به اسم اینترنت می‌شناسیم، حاصل همین توسعه تدریجی و اتصال شبکه‌های مختلف به یکدیگه است.

پس اگر بخوام خیلی ساده بگم:

اینترنت یک شبکه واحد و متمرکز نیست؛ مجموعه عظیمی از شبکه‌های کامپیوتریه که با استفاده از استانداردهای مشترک با هم ارتباط برقرار می‌کنن.

یعنی کامپیوتر من در تهران می‌تونه با کامپیوتری در یک کشور دیگه ارتباط برقرار کنه، چون هر دو از قواعد مشترکی برای ارسال و دریافت اطلاعات استفاده می‌کنن.»

برو اسلاید 5

بخش چهارم: اینترنت در دنیای واقعی چه شکلی است؟ (۴ دقیقه)

«حالا که فهمیدیم اینترنت مجموعه‌ای از شبکه‌های متصله، بیایید ببینیم این ارتباط در دنیای واقعی چطور برقرار می‌شه.

ما وقتی از اینترنت استفاده می‌کنیم، معمولاً با گوشی یا لپ‌تاپ به یک شبکه بی‌سیم وصل می‌شیم و همه‌چیز خیلی راحت اتفاق می‌افته.

اما پشت این تجربه ساده، یک زیرساخت فیزیکی بسیار بزرگ وجود داره.

بخش قابل توجهی از اطلاعاتی که بین قاره‌ها منتقل می‌شن، از طریق کابل‌های فیبر نوری زیردریایی جابه‌جا می‌شن.

برو اسلاید 6
بله، واقعاً کف اقیانوس‌ها کابل‌هایی وجود داره که قاره‌ها رو به هم متصل می‌کنن!

فیبر نوری رشته بسیار ظریفی از شیشه یا مواد شفاف دیگه است که اطلاعات رو به شکل پالس‌های نوری منتقل می‌کنه.
این فناوری امکان انتقال حجم عظیمی از اطلاعات رو با سرعت بسیار بالا در مسافت‌های طولانی فراهم می‌کنه.

البته همه ارتباطات اینترنتی از طریق فیبر نوری نیستن. در بعضی مسیرها از کابل‌های مسی، ارتباطات رادیویی، شبکه‌های موبایل و ماهواره‌ها هم استفاده می‌شه.

مثلاً وقتی گوشی شما با وای‌فای به مودم متصل می‌شه، ارتباط بین گوشی و مودم از طریق امواج رادیویی برقرار می‌شه. اما خود مودم معمولاً از طریق یک اتصال دیگه به شبکه ارائه‌دهنده اینترنت متصل شده.

یا وقتی از اینترنت موبایل استفاده می‌کنید، گوشی شما با دکل‌های مخابراتی ارتباط برقرار می‌کنه و از اونجا وارد شبکه بزرگ‌تر ارتباطی می‌شه.

پس اینترنت یک فضای جادویی و نامرئی نیست. بخش بزرگی از اون زیرساخت فیزیکی داره؛ از کابل‌های داخل ساختمان گرفته تا مراکز مخابراتی، تجهیزات مسیریابی، مراکز داده و کابل‌های عظیم کف اقیانوس‌ها.

رفتن به سایت https://www.submarinecablemap.com

«این سایت نقشه‌ای از کابل‌های زیردریایی ارتباطیه. هرکدوم از این خطوط، مسیر یک یا چند کابل فیبر نوری رو نشون می‌دن.

جالبه که بدونید این کابل‌ها فقط برای اینترنت نیستن؛ بخش بزرگی از ارتباطات بین‌المللی داده و مخابرات رو هم پشتیبانی می‌کنن.

بنابراین وقتی از یک وب‌سایت در یک کشور دیگه اطلاعات دریافت می‌کنیم، ممکنه بخشی از مسیر اطلاعات از همین کابل‌ها عبور کرده باشه.»

برو اسلاید 7

بخش پنجم: اطلاعات چطور در اینترنت جابه‌جا می‌شوند؟ (۳ دقیقه)

«تا اینجا فهمیدیم کامپیوترها از طریق زیرساخت‌های فیزیکی به هم متصل هستن.

اما یک سؤال دیگه باقی می‌مونه.

فرض کنید من می‌خوام یک عکس با حجم ۱۰ مگابایت برای شما ارسال کنم.

آیا کل این عکس یک‌جا وارد شبکه می‌شه و از اون طرف خارج می‌شه؟

نه دقیقاً.

در بسیاری از ارتباطات شبکه‌ای، اطلاعات به قسمت‌های کوچک‌تری تقسیم می‌شن که بهشون Packet یا بسته اطلاعاتی می‌گیم.

بیایید با یک مثال ساده تصورش کنیم.

فرض کنید می‌خواید یک کتاب صد صفحه‌ای رو برای دوستتون بفرستید، اما امکان ارسال یک‌جای کتاب وجود نداره.

میاید کتاب رو به چند قسمت تقسیم می‌کنید، هر قسمت رو جداگانه بسته‌بندی می‌کنید و برای دوستتون می‌فرستید.

دوستتون بسته‌ها رو دریافت می‌کنه و در نهایت دوباره اون‌ها رو کنار هم قرار می‌ده تا کتاب کامل بشه.

انتقال اطلاعات در شبکه هم از یک منطق مشابه استفاده می‌کنه.

داده‌ها به بسته‌های کوچک‌تری تقسیم می‌شن و هر بسته، اطلاعات لازم برای انتقال و شناسایی مقصد رو همراه خودش داره.

حالا چه کسی تصمیم می‌گیره این بسته‌ها از چه مسیری به مقصد برسن؟

برو اسلاید ۸

اینجاست که دستگاهی به نام Router یا روتر وارد ماجرا می‌شه.

روترها در شبکه وظیفه مسیریابی بسته‌های اطلاعاتی رو بر عهده دارن. یعنی با توجه به اطلاعات مقصد و وضعیت مسیرهای موجود، تصمیم می‌گیرن بسته‌ها رو به کدوم مسیر بعدی هدایت کنن.

مثلاً وقتی شما از کامپیوتر خودتون به یک سرور در کشور دیگه متصل می‌شید، اطلاعات ممکنه از چندین شبکه و روتر مختلف عبور کنه تا به مقصد برسه.

نکته جالب اینه که لزوماً همه بسته‌های یک ارتباط از یک مسیر عبور نمی‌کنن. بسته به شرایط شبکه و مسیریابی، ممکنه مسیرها متفاوت باشن.

در مقصد هم اطلاعات دریافت‌شده دوباره به شکلی که برنامه موردنظر نیاز داره بازسازی و پردازش می‌شن.

پس اگر بخوام این بخش رو جمع‌بندی کنم:

- اطلاعات در شبکه به شکل داده‌های دیجیتال منتقل می‌شن.
- در بسیاری از ارتباطات، داده‌ها به بسته‌های کوچک‌تر تقسیم می‌شن.
- این بسته‌ها از طریق زیرساخت‌های فیزیکی شبکه حرکت می‌کنن.
- روترها وظیفه هدایت اون‌ها به سمت مقصد رو بر عهده دارن.
- در مقصد، اطلاعات دریافت و پردازش می‌شن.

حالا ما به یک درک اولیه از چیزی که پشت پرده اینترنت اتفاق می‌افته رسیدیم.»

برو اسلاید 9

جمع‌بندی و پل ارتباطی به بخش بعدی (۱ دقیقه)

«خب، تا اینجا فهمیدیم اینترنت چیه و چطور کار می‌کنه.

فهمیدیم که اینترنت مجموعه‌ای از شبکه‌های به‌هم‌پیوسته است که با استفاده از زیرساخت‌های فیزیکی و قواعد ارتباطی مشترک، امکان انتقال اطلاعات رو فراهم می‌کنن.

اما هنوز یک سؤال مهم داریم.

برو اسلاید 10

ما تا اینجا درباره ارتباط کامپیوترها و انتقال اطلاعات صحبت کردیم. ولی وقتی شما مرورگرتون رو باز می‌کنید و وارد یک وب‌سایت می‌شید، دقیقاً از کدوم قسمت این شبکه استفاده می‌کنید؟

اصلاً وب‌سایت چه ارتباطی با اینترنت داره؟

آیا اینترنت و وب یک چیز هستن؟

این دقیقاً موضوع بخش بعدیه.»

بلوک ۲: تفاوت اینترنت و وب؛ از یک زیرساخت تا دنیایی از صفحه‌ها

اسلاید ۱۱

خب، توی بخش قبلی دیدیم که اینترنت در واقع یه شبکه‌ی خیلی بزرگ از شبکه‌هاییه که به هم وصل شدن. از کابل و فیبر نوری و دکل‌های مخابراتی گرفته تا روترها و مراکز داده، همه‌ی این‌ها کنار هم کار می‌کنن تا دستگاه‌های مختلف بتونن با هم ارتباط برقرار کنن.

حالا یه سؤال ساده: وقتی می‌گیم «اینترنت»، دقیقاً منظورمون چیه؟ و وقتی می‌گیم «وب‌سایت»، آیا داریم درباره‌ی همون اینترنت حرف می‌زنیم؟

ما توی صحبت‌های روزمره معمولاً این دو تا کلمه رو به جای هم استفاده می‌کنیم. مثلاً می‌گیم «اینترنت قطع شده»، در حالی که ممکنه فقط دسترسی ما به وب‌سایت‌ها دچار مشکل شده باشه، ولی پیام‌رسان یا یه سرویس دیگه هنوز کار کنه.

یا مثلاً می‌گیم «رفتم توی اینترنت»، در حالی که در واقع وارد وب شدیم و داریم از طریق مرورگر وب‌گردی می‌کنیم.

این دو تا مفهوم به هم وابسته‌ان، ولی در واقع یکی نیستن.

برو اسلاید ۱۲

یه تشبیه ساده وجود داره که کمک می‌کنه تفاوت این دو تا رو بهتر متوجه بشیم.

بیایم اینترنت رو مثل یه شبکه‌ی بزرگ از جاده‌ها و زیرساخت حمل‌ونقل در نظر بگیریم. وب هم یکی از سرویس‌هاییه که از این زیرساخت استفاده می‌کنه.

مثلاً تصور کنید جاده‌ها وجود دارن و روی این جاده‌ها اتوبوس، کامیون، تاکسی و ماشین‌های مختلف در حال رفت‌وآمدن.

اینترنت هم دقیقاً نقش همون زیرساخت ارتباطی و جاده‌ها رو داره.

اما وب یا همون World Wide Web، وب، اتوبوسه‌س که توی جاده حرکت میکنه. یعنی در واقع یه سامانه برای دسترسی به منابعیه که به همدیگه پیوند دارن؛ منابعی مثل صفحه‌های وب، تصویرها، ویدئوها و فایل‌هایی که معمولاً با استفاده از مرورگر بهشون دسترسی پیدا می‌کنیم.

ولی نکته اینجاست که وب تنها سرویسی نیست که از اینترنت استفاده می‌کنه.

ایمیل، تماس صوتی اینترنتی، بازی‌های آنلاین، انتقال فایل با FTP و خیلی از سرویس‌های دیگه هم از همین زیرساخت اینترنت استفاده می‌کنن.

پس ما توی شبکه‌ی اینترنت، با انواع مختلفی از داده‌ها سروکار داریم که باید از جایی به جای دیگه منتقل بشن. وب هم یکی از بسترها و سرویس‌هاییه که برای انتقال و دسترسی به این اطلاعات استفاده می‌شه.

بنابراین اگه وب رو کنار بذاریم، لزوماً خود اینترنت از بین نمی‌ره.

حالا برای اینکه مطمئن بشیم تفاوت این دو تا رو خوب متوجه شدیم، یه مثال بزنیم.

فرض کنید مرورگرتون رو باز می‌کنید و وارد یه سایت خبری می‌شید. اینجا دارید از وب استفاده می‌کنید.

حالا فرض کنید با یه برنامه‌ی تماس صوتی اینترنتی با یکی از دوستاتون صحبت می‌کنید. این ارتباط هم از اینترنت استفاده می‌کنه، ولی لزوماً هیچ وب‌گردی‌ای انجام نمی‌دید.

پس این جمله رو به خاطر بسپارید:

اینترنت زیرساخت ارتباطیه؛ وب یکی از سرویس‌هاییه که روی این زیرساخت کار می‌کنه.

برو اسلاید ۱۳

خب، حالا که فهمیدیم وب چیه، بیایم ببینیم اصلاً از کجا اومده و داستانش چی بوده.

سال ۱۹۸۹، یه دانشمند علوم کامپیوتر به اسم تیم برنرز، توی مرکز پژوهشی CERN در کشور سوئیس، یه پیشنهادی مطرح کرد.

پیشنهادش این بود که یه سامانه برای مدیریت و به‌اشتراک‌گذاری اطلاعات ایجاد بشه.

ماجرا از این قرار بود که پژوهشگرهایی که توی یه مجموعه‌ی بزرگ کار می‌کردن، با حجم زیادی از اطلاعات و مستندات سروکار داشتن و لازم بود بتونن خیلی راحت از یه سند به سند دیگه برن.

ایده‌ی اصلی وب هم دقیقاً از همین‌جا شکل گرفت: اینکه سندهای مختلف رو به همدیگه پیوند بدیم.

کل قضیه‌ی تولد وب همین بود!

یعنی به جای اینکه هر سند مثل یه جزیره‌ی جدا از بقیه باشه، می‌شد داخل یه سند، لینکی قرار داد که شما رو به یه سند دیگه هدایت کنه.

چیزی که امروز دیگه کاملاً برامون عادی شده. روی یه لینک کلیک می‌کنیم و وارد یه صفحه‌ی دیگه می‌شیم.

حالا رفته‌رفته توی سال‌های بعد، اجزای اصلی وب کم‌کم شکل گرفتن.

زبان HTML برای ساختار دادن به صفحه‌ها، سیستم آدرس‌دهی URL برای مشخص کردن منابع، و پروتکل HTTP برای درخواست و دریافت این منابع به وجود اومدن.

حالا شاید بگید اینایی که گفتم چی هستن؟ خواهیم دید :)

در نهایت، اولین وب‌سایت در سال ۱۹۹۱ در دسترس قرار گرفت. این وب‌سایت روی یه کامپیوتر NeXT در مرکز پژوهشی CERN میزبانی می‌شد.

دقیقه ۲۳ تا ۲۵ – برو اسلاید ۱۴

اسلاید ۱۴ – دموی زنده: info.cern.ch

خب، اگه بخوایم اون صفحه‌ی تاریخی رو ببینیم، می‌تونیم آدرس زیر رو باز کنیم:

اولین وب‌سایت تاریخ، یعنی info.cern.ch.

این صفحه‌ی تاریخی احتمالاً برای ما خیلی ساده و حتی یه مقدار عجیب به نظر می‌رسه.

نه خبری از تصاویر بزرگ و انیمیشن هست، نه منوهای پیچیده‌ای داره و نه از اون طراحی‌های جذاب و امروزی که الان توی سایت‌ها می‌بینیم.

اما چیزی که اهمیت داره، ظاهر این صفحه نیست؛ ایده‌ایه که پشتش قرار داشته. ایده‌ای که مسیر ارتباط و انتشار اطلاعات رو برای همیشه تغییر داد.

یه اتفاق مهم دیگه هم در سال ۱۹۹۳ افتاد. مرکز CERN اعلام کرد که فناوری وب رو بدون دریافت حق امتیاز در اختیار همه قرار می‌ده.

این تصمیم کمک کرد که وب به یه فناوری عمومی تبدیل بشه و امکان گسترش و استفاده از اون برای همه فراهم بشه.

وقتی به این صفحه نگاه می‌کنیم، به جای اینکه فقط به ظاهر ساده و قدیمی اون توجه کنیم، باید به یه نکته‌ی مهم دقت کنیم.

مهم‌ترین نوآوری این صفحه ظاهرش نبود؛ بلکه امکان پیوند دادن اطلاعات به یکدیگر بود.

همون قابلیتی که امروز تقریباً توی همه‌جا می‌بینیم؛ از وب‌سایت‌ها و مقاله‌ها گرفته تا فروشگاه‌های اینترنتی و شبکه‌های اجتماعی.

اسلایدهای ۱۵، ۱۶ و ۱۷ – نسل‌های مختلف وب

خب، گاهی برای توضیح تغییراتی که وب در طول زمان داشته، از اصطلاح‌هایی مثل Web 1.0، Web 2.0 و Web 3.0 استفاده می‌کنیم. احتمالاً به‌خصوص توی چند سال اخیر کلمه وب ۳ رو خیلی شنیده باشید.

فقط قبل از اینکه وارد این بحث بشیم، یه نکته‌ی مهم رو بگم.

این‌ها نسخه‌های رسمی یه نرم‌افزار نیستن که مثلاً یه روز نسخه‌ی یک تموم شده باشه و از فرداش همه به نسخه‌ی دو مهاجرت کرده باشن!

این اصطلاح‌ها بیشتر برای توصیف دوره‌های مختلف و روندهایی که توی هر دوره غالب بودن استفاده می‌شن. حتی مرز زمانی دقیق و مشخصی هم بین این دوره‌ها وجود نداره که همه روی اون توافق داشته باشن.

Web 1.0؛ وبی که بیشتر برای خوندن بود

اگه بخوایم یه تصویر کلی از Web 1.0 داشته باشیم، می‌تونیم بگیم وبی بود که بیشتر برای خوندن و دریافت اطلاعات استفاده می‌شد.

خیلی از وب‌سایت‌ها شبیه یه بروشور یا مجله‌ی دیجیتال بودن.

یعنی صاحب سایت محتوا رو منتشر می‌کرد و بازدیدکننده هم وارد سایت می‌شد و اون محتوا رو می‌خوند.

البته این‌طور هم نبود که توی اون دوره هیچ تعاملی وجود نداشته باشه. حتی اون زمان هم فرم‌هایی وجود داشتن که کاربر می‌تونست اطلاعاتی رو داخلشون وارد کنه.

پس نباید فکر کنیم که کاربران اصلاً نمی‌تونستن چیزی وارد سایت کنن.

اما به‌طور کلی، تولید و انتشار محتوا بیشتر در اختیار صاحب سایت یا همون مدیر سایت بود.

Web 2.0؛ وبی که کاربران هم توش مشارکت داشتن

بعد از اون، وارد دوره‌ای شدیم که معمولاً بهش Web 2.0 می‌گن.

توی این دوره، کاربرها دیگه فقط مصرف‌کننده‌ی محتوا نبودن؛ خودشون هم شروع کردن به تولید محتوا.

وبلاگ‌ها، شبکه‌های اجتماعی، سایت‌های اشتراک‌گذاری عکس و ویدئو، امکان ثبت نظر زیر مطالب و همکاری گروهی، همگی توی این دوره رشد زیادی پیدا کردن.

یعنی چی؟

یعنی کاربر دیگه فقط یه تماشاگر نبود؛ خودش هم می‌تونست توی فضای وب مشارکت کنه و محتوا تولید کنه. چیزی که شروعش رو به عنوان عصر وبلاگ‌نویسی توی ایران، اواسط 85 به بعد تا 94 اینا شاهدش بودیم.

Web 3.0؛ اصطلاحی با چند تا برداشت مختلف

اما Web 3.0 یه مقدار پیچیده‌تره.

چرا؟ چون این اصطلاح توی جاهای مختلف با معناهای متفاوتی استفاده می‌شه.

توی بعضی بحث‌ها، منظور از Web 3.0 همون وب معناییه؛ یعنی فضایی که توش داده‌ها طوری سازمان‌دهی می‌شن که ماشین‌ها هم بتونن معنا و ارتباط بین اطلاعات رو بهتر درک کنن.

اما توی سال‌های اخیر، این اصطلاح خیلی بیشتر برای صحبت درباره‌ی وب غیرمتمرکز، بلاک‌چین و مالکیت توزیع‌شده‌ی داده‌ها استفاده شده.

از طرف دیگه، هوش مصنوعی و عامل‌های هوشمند هم الان نقش پررنگی توی بحث‌های مربوط به آینده‌ی وب دارن.

البته بهتره همه‌ی این موضوعات رو توی یه تعریف قطعی و نهایی از Web 3.0 جا ندیم.

چون این حوزه هنوز در حال تغییره و هنوز توافق کاملی وجود نداره که دقیقاً منظور از این اصطلاح چیه.

پس برای این دوره، فعلاً این تصویر کلی رو توی ذهن داشته باشید:

وب در ابتدا بیشتر برای خوندن و دریافت اطلاعات استفاده می‌شد.

بعد کاربرها هم وارد ماجرا شدن و شروع کردن به مشارکت و تولید محتوا.

و امروز هم داریم تلاش‌هایی رو می‌بینیم برای اینکه وب هوشمندتر، متصل‌تر و در بعضی معماری‌ها غیرمتمرکزتر بشه.

اسلاید ۱۸ – جمع‌بندی و ورود به بحث بعدی

خب، تا اینجا متوجه شدیم که اینترنت و وب با هم فرق دارن و یکی نیستن.

دیدیم که وب توی مرکز پژوهشی CERN و با ایده‌ی پیوند دادن اطلاعات به همدیگه شکل گرفت.

همچنین دیدیم که تجربه‌ی استفاده از وب در طول زمان چقدر تغییر کرده؛ از صفحه‌های ساده‌ای که بیشتر برای خوندن و دریافت اطلاعات بودن، تا سرویس‌های تعاملی و پیچیده‌ای که امروز باهاشون سروکار داریم.

حالا می‌خوایم به یه سؤال مهم‌تر جواب بدیم.

فرض کنید آدرس یه سایت رو توی مرورگر وارد می‌کنیم و دکمه‌ی Enter رو می‌زنیم.

از همون لحظه به بعد، پشت صحنه چه اتفاقی می‌افته؟

این درخواستی که ما فرستادیم، چطوری مسیرش رو پیدا می‌کنه؟ اصلاً اون سایتی که می‌خوایم ببینیم کجا قرار داره؟ و اطلاعاتش چطوری به صفحه‌ی نمایش ما می‌رسه؟

برای اینکه به این سؤال‌ها جواب بدیم، می‌خوایم یه کلیک ساده رو از اول تا آخر دنبال کنیم و ببینیم پشت صحنه دقیقاً چه اتفاق‌هایی می‌افته.

بلوک سوم – پشت صحنه یک کلیک

اسلایدهای ۱۹ تا ۲۵

اسلاید ۱۹ – وقتی آدرس یک سایت رو وارد می‌کنیم، چه اتفاقی می‌افته؟

خب، تا اینجا فهمیدیم اینترنت چیه، وب چیه و این دو تا چطوری به هم ارتباط دارن.

حالا بیاید یه کار خیلی ساده انجام بدیم.

فرض کنید مرورگر رو باز می‌کنیم، آدرس `digikala.com` رو وارد می‌کنیم و دکمه‌ی Enter رو می‌زنیم.

از نگاه ما چه اتفاقی افتاده؟ یه آدرس نوشتیم و چند لحظه بعد، یه فروشگاه اینترنتی روی صفحه ظاهر شده.

اما پشت همین اتفاقی که از نظر ما خیلی ساده به نظر می‌رسه، چندین مرحله وجود داره که مرورگر و سیستم‌های مختلف باید انجام بدن.

اولین نکته‌ای که می‌خوام همین‌جا روشن کنم اینه که وقتی یه سایت رو باز می‌کنیم، لزوماً این‌طوری نیست که یه کامپیوتر اون طرف دنیا، کل صفحه رو آماده کنه و مثل یه عکس برامون بفرسته!

توی خیلی از وب‌سایت‌ها، سرور یه مجموعه از فایل‌ها و اطلاعات رو برای مرورگر می‌فرسته؛ مثلاً HTML، CSS، JavaScript، تصاویر و داده‌ها.

بعد مرورگر این منابع رو دریافت می‌کنه، اون‌ها رو پردازش می‌کنه و در نهایت صفحه رو می‌سازه.

البته همیشه هم ماجرا به همین شکل نیست. بعضی از وب‌سایت‌ها بخشی از صفحه رو از همون ابتدا روی خود سرور تولید می‌کنن و بعد برای مرورگر می‌فرستن. بعضی‌ها هم ترکیبی از این دو روش رو استفاده می‌کنن.

فعلاً لازم نیست وارد جزئیات فنی این روش‌ها بشیم.

چیزی که می‌خوام توی ذهنتون بمونه اینه که صفحه‌ای که روی مانیتور می‌بینیم، نتیجه‌ی همکاری چند بخش مختلفه؛ نه اینکه صرفاً یه تصویر باشه که از اینترنت دانلود شده.

خب، حالا برای اینکه بفهمیم مرورگر اصلاً چطوری مقصدش رو پیدا می‌کنه، باید با یه مفهوم آشنا بشیم: آدرس IP.

اسلاید ۲۰ – آدرس IP چیه؟

تصور کنید می‌خوایم برای یه نفر بسته‌ی پستی بفرستیم.

خب، طبیعتاً برای اینکه بسته به مقصد برسه، باید آدرس مقصد رو بدونیم.

توی شبکه‌های کامپیوتری هم دستگاه‌ها برای اینکه بتونن با هم ارتباط برقرار کنن، به آدرس نیاز دارن.

یکی از مهم‌ترین این آدرس‌ها، آدرس IP هست.

IP مخفف Internet Protocol هست و آدرس IP مشخص می‌کنه که یه دستگاه یا رابط شبکه، توی یه شبکه با چه نشونی قابل شناسایی و ارتباطه.

مثلاً یه آدرس IPv4 می‌تونه به این شکل باشه:

`192.0.2.10`

البته این عددها صرفاً یه نمونه‌ی آموزشی هستن که برای مستندات و مثال‌ها کنار گذاشته شدن. یعنی قرار نیست این آدرس رو توی مرورگرتون وارد کنید و انتظار داشته باشید به یه سایت واقعی برسید!

نکته اینجاست که ما آدما معمولاً با اسم‌ها راحت‌تریم تا یه رشته‌ی طولانی از عددها.

مثلاً اگه بخوام وارد یه فروشگاه اینترنتی بشم، ترجیح می‌دم بنویسم `digikala.com`، نه اینکه مجبور باشم یه آدرس عددی رو حفظ کنم.

اینجاست که دامنه‌ها وارد ماجرا می‌شن.

دامنه در واقع یه اسم قابل‌خوندن برای ما آدماست که از طریق سیستم DNS به اطلاعات لازم برای پیدا کردن سرویس موردنظر تبدیل می‌شه.

یه نکته‌ی جالب هم بگم: رابطه‌ی دامنه و IP همیشه یک‌به‌یک نیست.

یعنی ممکنه یه دامنه به چند تا آدرس IP مختلف اشاره کنه. از اون طرف هم ممکنه چند تا دامنه‌ی مختلف روی یه آدرس IP میزبانی بشن.

پس این تصور که هر سایت حتماً یه IP اختصاصی و ثابت داره، همیشه درست نیست.

حالا یه سؤال:

مرورگر از کجا می‌فهمه که `digikala.com` به چه آدرسی اشاره می‌کنه؟

پاسخ این سؤال DNS هست.

اسلاید ۲۱ – DNS؛ دفترچه تلفن اینترنت

DNS مخفف Domain Name System هست.

اگه بخوام خیلی ساده توضیحش بدم، DNS یه سیستم برای نام‌گذاری و پیدا کردن اطلاعات توی اینترنته که یکی از مهم‌ترین وظایفش اینه که آدرس IP مربوط به یه نام دامنه رو پیدا کنه.

تصور کنید شماره تلفن یه نفر رو ندارید، ولی اسمش رو می‌دونید.

چی کار می‌کنید؟ می‌رید سراغ دفترچه تلفن، اسمش رو پیدا می‌کنید و از روی اون، شماره تلفنش رو به دست میارید.

DNS هم از این نظر یه چیزی شبیه همین فرایندە.

وقتی توی مرورگر می‌نویسیم `digikala.com`، سیستم ما باید اطلاعات DNS مربوط به این دامنه رو پیدا کنه.

این کار معمولاً از طریق یه DNS Resolver انجام می‌شه.

این Resolver ممکنه توسط شرکت ارائه‌دهنده‌ی اینترنت، سیستم‌عامل، شبکه‌ی سازمانی یا حتی یه سرویس عمومی DNS در اختیار ما قرار گرفته باشه.

حالا اگه اطلاعاتی که لازم داریم از قبل توی حافظه‌ی موقت وجود نداشته باشه، Resolver می‌تونه از ساختار سلسله‌مراتبی DNS کمک بگیره.

مثلاً ممکنه اول بره سراغ سرورهای Root، بعد سرورهای مربوط به پسوند دامنه، مثلاً `.com`، و در نهایت سرور معتبر یا همون Authoritative مربوط به دامنه رو پیدا کنه.

این سرورها اطلاعات موردنیاز رو در اختیار Resolver قرار می‌دن و نتیجه هم می‌تونه برای یه مدتی ذخیره بشه تا دفعه‌های بعد سریع‌تر به دست بیاد.

البته این‌طوری نیست که این فرایند همیشه از اول تا آخر تکرار بشه؛ چون ممکنه اطلاعات موردنیاز از قبل توی حافظه‌های موقت مختلف ذخیره شده باشن.

یه نکته‌ی دیگه هم وجود داره.

DNS فقط برای تبدیل نام دامنه به IP استفاده نمی‌شه. انواع دیگه‌ای از اطلاعات هم توی DNS نگهداری می‌شن؛ مثلاً اطلاعات مربوط به سرورهای ایمیل یه دامنه.

پس DNS رو فقط یه دفترچه تلفن ساده در نظر نگیرید؛ در واقع DNS یه سیستم توزیع‌شده برای نگهداری و بازیابی اطلاعات مربوط به نام دامنه‌هاست.

خب، حالا که مقصد رو پیدا کردیم، باید ببینیم درخواست ما چطوری به اون مقصد می‌رسه و چه چیزی از اون طرف برمی‌گرده.

اسلاید ۲۲ – Client و Server؛ درخواست از یه طرف، پاسخ از طرف دیگه

اینجا با دو تا مفهوم مهم روبه‌رو می‌شیم: Client و Server.

اول Client یا همون کارخواه به زبان فارسی.

Client در واقع یه نرم‌افزار یا دستگاهیه که درخواست می‌فرسته و از یه سرویس استفاده می‌کنه.

مثلاً وقتی با مرورگرتون وارد یه سایت می‌شید، مرورگر شما نقش Client رو داره.

اون طرف ماجرا هم Server یا سرویس‌دهنده قرار داره.

سرور سیستمیه که درخواست‌ها رو دریافت می‌کنه و بر اساس وظیفه‌ای که براش تعریف شده، به اون‌ها پاسخ می‌ده.

مثلاً وقتی وارد یه فروشگاه اینترنتی می‌شید، مرورگر شما یه درخواست می‌فرسته و سرور فروشگاه هم اطلاعاتی رو که لازم دارید براتون برمی‌گردونه.

این ارتباط توی وب معمولاً با استفاده از پروتکل HTTP انجام می‌شه.

HTTP یه پروتکل ارتباطیه که مشخص می‌کنه درخواست‌ها و پاسخ‌های وب باید چه ساختاری داشته باشن.

مثلاً مرورگر می‌تونه درخواست بده که صفحه‌ی اصلی یه سایت رو دریافت کنه. سرور هم ممکنه در پاسخ، کد HTML اون صفحه رو برگردونه.

یا مثلاً مرورگر می‌تونه از یه سرویس درخواست کنه که فهرست محصولات رو با قالب JSON تحویل بده.

پس این‌طور نیست که پاسخ سرور همیشه یه صفحه‌ی HTML باشه. ممکنه چیزی که برمی‌گرده تصویر، فایل، داده یا هر نوع منبع دیگه‌ای باشه.

احتمالاً اسم HTTPS رو هم زیاد دیده باشید.

HTTPS در واقع همون استفاده از HTTP روی یه ارتباط رمزنگاری‌شده با استفاده از TLS هست.

این ارتباط کمک می‌کنه اطلاعاتی که بین ما و سرور ردوبدل می‌شن، به‌سادگی توسط افراد دیگه قابل خوندن یا دست‌کاری نباشن. علاوه بر این، مرورگر هم می‌تونه هویت سرور رو بررسی کنه.

به همین دلیله که امروزه استفاده از HTTPS برای وب‌سایت‌ها اهمیت خیلی زیادی داره.

خب، پس تا اینجا یه مسیر کلی رو داریم:

ما یه نام دامنه وارد می‌کنیم، DNS اطلاعات لازم برای پیدا کردن مقصد رو فراهم می‌کنه، مرورگر به سرور درخواست می‌فرسته و سرور هم پاسخ رو برمی‌گردونه.

اما آیا همه‌ی سایت‌ها دقیقاً به یه شکل کار می‌کنن؟

نه!

اسلاید ۲۳ – سایت استاتیک و داینامیک چه فرقی دارن؟

احتمالاً تا حالا اصطلاح‌های Static Website و Dynamic Website رو شنیدید.

بیاید با یه مثال ساده ببینیم این دو تا چه فرقی با هم دارن.

فرض کنید یه وب‌سایت داریم که فقط چند تا صفحه برای معرفی یه شرکته.

متن‌ها و تصاویرش از قبل آماده شدن و هر کسی هم که وارد سایت بشه، تقریباً همون محتوای مشخص رو می‌بینه.

این یه نمونه از وب‌سایت استاتیکه.

توی همچین سایتی، فایل‌های HTML و CSS و تصاویر معمولاً از قبل آماده شدن و سرور همون فایل‌ها رو برای مرورگر می‌فرسته.

حالا بیاید یه مثال دیگه رو در نظر بگیریم.

فرض کنید وارد یه فروشگاه اینترنتی می‌شید.

شما وارد حساب کاربری خودتون شدید، چند تا محصول توی سبد خریدتون دارید و قیمت‌ها هم بر اساس اطلاعات فعلی فروشگاه به شما نمایش داده می‌شن.

حالا ممکنه یه کاربر دیگه با حساب کاربری خودش وارد همون سایت بشه و اطلاعات کاملاً متفاوتی رو ببینه.

اینجا محتوای صفحه می‌تونه بر اساس کاربر، اطلاعات پایگاه داده، وضعیت سفارش‌ها یا شرایط دیگه تغییر کنه.

این نوع رفتار رو معمولاً توی وب‌سایت‌های داینامیک می‌بینیم. یعنی محتوای دیده شده و مورد استفاده توسط دوتا کاربر می‌تونه با همدیگه متفاوت باشه.

حالا یه اشتباه رایج رو همین‌جا اصلاح کنیم.

استاتیک بودن یه سایت به این معنی نیست که سایت هیچ حرکتی نداره یا نمی‌شه باهاش تعامل داشت!

ممکنه یه سایت استاتیک انیمیشن داشته باشه، منوهای تعاملی داشته باشه و حتی با JavaScript به کلیک‌های کاربر واکنش نشون بده.

پس تفاوت اصلی این دو تا توی اینه که محتوای سایت چطوری تولید و ارائه می‌شه، نه اینکه صفحه چقدر متحرک یا جذابه.

و اینکه توی دنیای واقعی، خیلی از وب‌سایت‌ها ترکیبی از هر دو روش رو استفاده می‌کنن.

اسلاید ۲۴ – اجرای دمو: سفر یه درخواست توی اینترنت

خب، حالا می‌خوام یه نمونه‌ی تصویری بهتون نشون بدم تا چیزهایی که تا اینجا گفتیم، یه مقدار ملموس‌تر بشن.

من یه انیمیشن آماده کردم که توش چند تا شهر مختلف رو روی کره‌ی زمین می‌بینیم؛ مثلاً تهران، دبی، استانبول و فرانکفورت.

توی این مثال فرض می‌کنیم یه کاربر توی تهران می‌خواد یه وب‌سایت رو باز کنه و سرور اون وب‌سایت هم توی یه نقطه‌ی دیگه قرار داره.

قبل از اینکه دکمه‌ها رو بزنیم، یه نکته‌ی مهم رو بگم.

این انیمیشن در واقع یه مدل آموزشی و ساده‌شده از ارتباطات اینترنتیه. مسیر واقعی بسته‌های اطلاعاتی توی اینترنت لزوماً به همین شکلی که اینجا می‌بینیم نیست.

این مسیر ممکنه با توجه به شرکت ارائه‌دهنده‌ی اینترنت، مسیریابی شبکه، محل سرورها و اطلاعاتی که توی حافظه‌های موقت ذخیره شدن، تغییر کنه.

یه نکته‌ی دیگه هم اینکه DNS قرار نیست حتماً درخواست ما رو به یه شهر خاص، مثلاً دبی، بفرسته. چیزی که اینجا می‌بینیم صرفاً برای اینه که مراحل کار رو به شکل تصویری بهتر متوجه بشیم.

خب، بریم سراغ دمو.

[روی دکمه «۱. پرسش از DNS» کلیک کن]

خب، می‌بینید که یه بسته‌ی زردرنگ از سمت کاربر ارسال می‌شه.

این مرحله در واقع نشون‌دهنده‌ی درخواست برای پیدا کردن اطلاعات مربوط به نام دامنه‌ست.

یعنی سیستم ما می‌خواد بدونه برای اینکه بتونه با سرویس موردنظر ارتباط برقرار کنه، باید از چه آدرسی استفاده کنه.

حالا بریم سراغ دکمه‌ی دوم.

[روی دکمه «۲. ارسال درخواست به سرور» کلیک کن]

اینجا درخواست ما به سمت سرور ارسال می‌شه.

می‌بینید که یه بسته‌ی آبی‌رنگ توی مسیر حرکت می‌کنه.

حالا توی سمت سرور، درخواست دریافت می‌شه و برنامه‌ای که روی سرور در حال اجراست، شروع به پردازش اون می‌کنه.

[بخش ترمینال سرور رو نشون بده و در صورت امکان Slow Motion رو فعال کن]

اینجا می‌تونیم یه نکته‌ی مهم رو ببینیم.

سرور فقط یه دستگاه خاموش نیست که یه سری فایل داخلش ذخیره شده باشه!

ممکنه نرم‌افزارهای مختلفی روی سرور در حال اجرا باشن که درخواست‌ها رو دریافت و پردازش می‌کنن.

مثلاً ممکنه یه برنامه با Python یا Node.js نوشته شده باشه.

این برنامه می‌تونه اطلاعات موردنیازش رو از پایگاه داده بخونه، یه سری محاسبات انجام بده یا محتوایی رو که لازمه آماده کنه.

خب، حالا نوبت پاسخ سروره.

[روی دکمه «۳. اجرای برنامه و بازگشت کد» کلیک کن]

این بار یه بسته‌ی سبزرنگ رو می‌بینیم که داره به سمت کاربر برمی‌گرده.

توی این مثال، پاسخ سرور شامل منابعیه که مرورگر برای نمایش صفحه به اون‌ها نیاز داره.

مرورگر این اطلاعات رو دریافت می‌کنه و صفحه رو می‌سازه.

و در نهایت، چیزی که کاربر می‌بینه یه فروشگاه اینترنتیه.

همین!

یه اتفاق چندمرحله‌ای توی پشت صحنه افتاده، ولی از دید کاربر فقط یه آدرس وارد شده و یه صفحه باز شده.

اسلاید ۲۵ – جمع‌بندی بلوک سوم

خب، بیاید یه بار دیگه چیزهایی رو که تا اینجا یاد گرفتیم کنار هم بذاریم.

وقتی یه آدرس وب‌سایت رو توی مرورگر وارد می‌کنیم، اول از همه باید اطلاعات مربوط به مقصد رو پیدا کنیم.

نام دامنه برای ما آدما قابل‌خوندن‌تره و DNS کمک می‌کنه اطلاعات لازم برای پیدا کردن مقصد رو به دست بیاریم.

بعد مرورگر ما که نقش Client رو داره، با استفاده از پروتکل‌هایی مثل HTTP یا HTTPS یه درخواست به سرور می‌فرسته.

سرور هم درخواست رو پردازش می‌کنه و پاسخ رو برمی‌گردونه.

این پاسخ ممکنه شامل HTML، CSS، JavaScript، تصاویر یا اطلاعاتی با قالب‌هایی مثل JSON باشه.

در نهایت مرورگر این منابع رو پردازش می‌کنه و چیزی رو که ما به عنوان یه صفحه‌ی وب می‌بینیم، روی صفحه‌ی نمایش نشون می‌ده.

خب، حالا یه سؤال جدید داریم:

این مرورگری که این همه کار انجام می‌ده، دقیقاً چه وظایفی داره؟

و اصلاً خود صفحه‌ی وب چطوری ساخته می‌شه؟

برای اینکه به این سؤال جواب بدیم، باید یه مقدار با اجزای اصلی طراحی و توسعه‌ی وب آشنا بشیم.

بلوک چهارم – اجزای سازنده صفحات وب

اسلایدهای ۲۶ تا ۳۲ | دقیقه ۵۰ تا ۷۵ کلاس

اسلاید ۲۶ – مرورگر؛ فقط یه پنجره برای دیدن اینترنت نیست!

خب، تا اینجا چند بار اسم مرورگر رو شنیدیم.

اما مرورگر دقیقاً چه کاری انجام می‌ده؟

مرورگر یا Web Browser در واقع یه نرم‌افزاره که به ما اجازه می‌ده با منابع و سرویس‌های وب ارتباط برقرار کنیم.

مرورگرهایی مثل Safari، Firefox، Chrome و Edge رو احتمالاً همه‌تون می‌شناسید.

اما کار مرورگر فقط این نیست که یه صفحه رو از اینترنت بگیره و به ما نشون بده.

مرورگر باید درخواست‌ها رو ارسال کنه، پاسخ‌ها رو دریافت کنه، فایل‌های مختلف رو پردازش کنه و در نهایت اون‌ها رو به شکلی در اختیار ما قرار بده که قابل مشاهده و استفاده باشن.

مثلاً وقتی یه فایل HTML دریافت می‌کنه، ساختار اون رو بررسی می‌کنه.

وقتی فایل CSS رو دریافت می‌کنه، ازش برای مشخص کردن ظاهر و چیدمان عناصر صفحه استفاده می‌کنه.

وقتی هم JavaScript اجرا می‌شه، ممکنه قابلیت‌های تعاملی صفحه فعال بشن.

مرورگر کارهای دیگه‌ای هم انجام می‌ده؛ مثلاً کوکی‌ها رو مدیریت می‌کنه، اطلاعات رو به‌صورت محلی ذخیره می‌کنه، کدهای JavaScript رو اجرا می‌کنه، درخواست‌های شبکه رو مدیریت می‌کنه و دسترسی‌های امنیتی رو کنترل می‌کنه.

یعنی در واقع مرورگر یه محیط اجرایی نسبتاً پیچیده‌ست.

یه نکته‌ی جالب دیگه هم وجود داره.

مرورگرهای مختلف ممکنه توی جزئیات پردازش و نمایش بعضی قابلیت‌ها با هم تفاوت داشته باشن.

به همین دلیله که یه توسعه‌دهنده‌ی وب باید مطمئن بشه وب‌سایتی که ساخته، توی مرورگرهای مختلف به‌درستی کار می‌کنه.

پس وقتی می‌گیم یه صفحه‌ی وب توی مرورگر نمایش داده می‌شه، منظورمون این نیست که مرورگر فقط یه عکس رو باز کرده!

بلکه یه مجموعه از فایل‌ها و دستورالعمل‌ها رو پردازش کرده و نتیجه‌شون رو به ما نشون داده.

خب، حالا بیاید ببینیم اصلاً چه کسانی این صفحه‌ها رو طراحی و تولید می‌کنن.

اسلاید ۲۷ – طراحی وب با توسعه وب چه فرقی داره؟

وقتی صحبت از ساخت یه وب‌سایت می‌شه، معمولاً دو تا اصطلاح رو می‌شنویم: Web Design و Web Development.

این دو تا حوزه به هم مربوطن، ولی دقیقاً یه کار رو انجام نمی‌دن.

Web Design یا طراحی وب، بیشتر به این موضوع می‌پردازه که یه وب‌سایت چه شکلی داشته باشه و کاربر چطوری باهاش ارتباط برقرار کنه.

مثلاً اینکه رنگ‌ها چه‌طوری باشن، دکمه‌ها کجا قرار بگیرن، منوها چطوری طراحی بشن، اندازه‌ی متن‌ها چقدر باشه و کاربر چطوری بتونه به‌راحتی به اطلاعاتی که نیاز داره دسترسی پیدا کنه.

توی این حوزه، مفاهیمی مثل طراحی رابط کاربری یا UI و تجربه‌ی کاربری یا UX اهمیت زیادی دارن.

از اون طرف، Web Development یا توسعه‌ی وب، مربوط به پیاده‌سازی فنی وب‌سایته.

یعنی اینکه چطوری اون طرح رو به یه صفحه‌ی واقعی تبدیل کنیم، چطوری دکمه‌ها رو فعال کنیم، اطلاعات رو دریافت و ذخیره کنیم و بخش‌های مختلف سیستم رو به هم وصل کنیم.

البته توی دنیای واقعی، مرز این دو تا حوزه همیشه کاملاً مشخص نیست.

ممکنه یه نفر هم طراحی انجام بده و هم برنامه‌نویسی کنه. ممکنه یه تیم داشته باشیم که توش طراح‌ها و توسعه‌دهنده‌ها جدا از هم کار کنن.

حتی ابزارهایی هم وجود دارن که بخشی از فاصله‌ی بین طراحی و پیاده‌سازی رو کم می‌کنن.

اما نکته اینجاست که برای ساختن یه صفحه‌ی وب، صرفاً داشتن یه طرح زیبا کافی نیست. باید بتونیم اون طرح رو با استفاده از فناوری‌های وب پیاده‌سازی کنیم.

و اینجاست که می‌رسیم به سه تا ابزار پایه‌ای و خیلی مهم در توسعه‌ی صفحات وب: HTML، CSS و JavaScript.

اسلاید ۲۸ – HTML، CSS و JavaScript؛ سه جزء اصلی صفحات وب

اگه بخوایم ساختار یه صفحه‌ی وب رو خیلی ساده توضیح بدیم، معمولاً با سه تا فناوری اصلی سروکار داریم.

اول HTML.

HTML مخفف HyperText Markup Language هست. سه تا بخش اصلی داره. اولاً زبانه. دوماً نشانه گذاریه. سوماً فرامتنیه. یعنی توش تصویر میتونی استفاده کنی. فایل مدیا مثل ویدیو و فایل صوتی و... میتونی بذاری. فرم و دکمه و... میتونی استفاده کنی. پس فرا متنیه.

HTML یه زبان نشانه‌گذاریه و وظیفه‌ی اصلیش اینه که ساختار و معنای محتوای صفحه رو مشخص کنه.

مثلاً به مرورگر می‌گیم این قسمت یه عنوانه، این بخش یه پاراگرافه، اینجا یه تصویر قرار داره و این قسمت هم یه لینکه.

در واقع HTML مشخص می‌کنه چه عناصری توی صفحه وجود دارن و از نظر ساختاری چه ارتباطی با هم دارن.

دوم CSS.

CSS مخفف Cascading Style Sheets هست.

CSS مسئول مشخص کردن ظاهر و چیدمان عناصر صفحه‌ست.

مثلاً مشخص می‌کنیم عنوان چه اندازه‌ای داشته باشه، رنگ پس‌زمینه چی باشه، فاصله‌ی بین بخش‌های مختلف چقدر باشه، عناصر کنار هم قرار بگیرن یا زیر هم و صفحه توی موبایل چه شکلی نمایش داده بشه.

پس CSS فقط برای قشنگ‌تر کردن صفحه نیست!

بخش مهمی از چیدمان، خوانایی و سازگاری صفحه با اندازه‌های مختلف نمایشگر رو کنترل می‌کنه.

و سوم JavaScript.

JavaScript زبان برنامه‌نویسی اصلی سمت مرورگره که به ما اجازه می‌ده رفتار و منطق تعاملی صفحه رو پیاده‌سازی کنیم.

مثلاً وقتی روی یه دکمه کلیک می‌کنیم، یه اطلاعاتی نمایش داده بشه، یه فرم بررسی بشه، قیمت یه محصول بر اساس انتخاب کاربر تغییر کنه یا اطلاعات جدیدی از سرور دریافت بشه.

حالا اگه بخوایم یه مثال ساده بزنیم:

HTML مشخص می‌کنه چه چیزهایی توی صفحه وجود داشته باشن.

CSS مشخص می‌کنه این چیزها چطوری نمایش داده بشن.

JavaScript هم مشخص می‌کنه صفحه چطوری رفتار کنه و چطوری به تعاملات کاربر پاسخ بده.

امروزه این سه تا فناوری توی دنیای واقعی کاملاً از هم جدا نیستن و با هم همکاری می‌کنن.

مثلاً JavaScript می‌تونه ساختار HTML رو تغییر بده و CSS هم می‌تونه بر اساس وضعیت عناصر، ظاهر متفاوتی به اون‌ها بده.

اما همین تقسیم‌بندی اولیه برای شروع یادگیری کاملاً مناسبه.

اسلاید ۲۹ – Frontend و Backend؛ دو سمت یه وب‌سایت

خب، یه اصطلاح دیگه هم وجود داره که احتمالاً زیاد شنیدید: Frontend و Backend.

Frontend یا بخش جلویی، همون قسمتی از وب‌سایته که کاربر مستقیماً باهاش سروکار داره.

مثلاً صفحه‌ای که باز می‌کنید، دکمه‌هایی که می‌بینید، فرم‌هایی که پر می‌کنید، منوها و اطلاعاتی که روی صفحه نمایش داده می‌شن.

اگه بخوایم خیلی ساده بگیم، Frontend همون بخشی از سایته که توی مرورگر کاربر اجرا و نمایش داده می‌شه.

اما Backend یا بخش پشتی، مربوط به منطق و پردازش‌هاییه که پشت صحنه انجام می‌شن. دقیقاً اون نرم افزاری که توی سرور اجرا میشه. میشه گفت بک‌اند، مسئول اینه که به مرورگر ما بگه که توی فرانت‌اند، چه چیزهایی بهمون نمایش داده بشن.

برای مثال، فرض کنید وارد یه فروشگاه اینترنتی شدید.

یه محصول رو انتخاب می‌کنید و روی دکمه‌ی افزودن به سبد خرید کلیک می‌کنید.

چیزی که شما می‌بینید، مثلاً تغییر عددی که کنار آیکون سبد خرید نمایش داده می‌شه، بخشی از Frontend هست.

اما ممکنه اطلاعات سبد خرید توی حساب کاربری شما روی سرور ذخیره بشه، قیمت‌ها دوباره بررسی بشن، موجودی محصول کنترل بشه و اطلاعات سفارش به پایگاه داده منتقل بشه.

این کارها می‌تونن توی Backend انجام بشن.

در واقع Frontend و Backend با هم ارتباط دارن.

Frontend درخواست می‌فرسته و Backend اطلاعات موردنیاز رو پردازش می‌کنه و پاسخ می‌ده.

این ارتباط معمولاً از طریق API انجام می‌شه؛ مفهومی که توی جلسات بعدی بیشتر باهاش آشنا می‌شیم.

یه نکته‌ی مهم هم اینجا وجود داره:

Backend لزوماً فقط یه سرور و یه پایگاه داده نیست.

ممکنه چندین سرویس، نرم‌افزار، سیستم ذخیره‌سازی و ابزار مختلف پشت صحنه‌ی یه وب‌سایت در حال فعالیت باشن.

اما فعلاً همین تصویر کلی رو توی ذهنتون داشته باشید:

Frontend همون بخشیه که کاربر باهاش کار می‌کنه و Backend بخش مهمی از منطق و پردازش‌های پشت صحنه رو انجام می‌ده.

اسلاید ۳۰ – دمو: بازرسی یه صفحه‌ی وب توی مرورگر

خب، حالا می‌خوام یه قابلیت جالب از مرورگر رو بهتون نشون بدم.

تا اینجا درباره‌ی HTML و CSS صحبت کردیم.

اما شاید این سؤال براتون پیش اومده باشه که اصلاً از کجا می‌تونیم بفهمیم یه صفحه‌ی وب واقعاً از چه چیزهایی تشکیل شده؟

مرورگرها یه ابزاری دارن به اسم Developer Tools.

این ابزار برای توسعه‌دهنده‌ها طراحی شده و امکانات مختلفی در اختیارشون قرار می‌ده تا بتونن صفحه‌های وب رو بررسی کنن و مشکلاتشون رو پیدا کنن.

خب، من الان یه وب‌سایت رو باز می‌کنم.

[یه وب‌سایت مناسب رو باز کن و کل صفحه رو از طریق اشتراک‌گذاری صفحه نمایش بده]

حالا کلید F12 رو می‌زنم.

می‌بینید که یه پنجره یا بخش جدید باز می‌شه. این همون Developer Tools هست.

توی این ابزار چند تا تب مختلف وجود داره، ولی فعلاً می‌خوایم با تب Elements کار کنیم.

[تب Elements رو انتخاب کن]

اینجا می‌تونیم ساختار HTML صفحه رو ببینیم.

مثلاً روی یکی از عنوان‌ها یا تصاویر صفحه کلیک می‌کنم. می‌بینید که مرورگر قسمت مربوط به همون عنصر رو توی ساختار HTML مشخص می‌کنه.

حالا می‌تونیم بریم سراغ قسمت Styles.

[توی بخش Styles یه ویژگی CSS رو تغییر بده؛ مثلاً رنگ متن یا اندازه‌ی فونت]

ببینید! فقط با تغییر یه مقدار، ظاهر صفحه عوض شد.

حالا ممکنه این سؤال براتون پیش بیاد که یعنی من تونستم وب‌سایت اصلی رو تغییر بدم؟

نه!

این تغییری که انجام دادیم، فقط توی مرورگر خود من اتفاق افتاده.

در واقع من فقط نسخه‌ای رو که مرورگر دریافت کرده، به‌صورت موقت دست‌کاری کردم.

این تغییرات نه به سرور اصلی ارسال می‌شن و نه توی خود وب‌سایت ذخیره می‌شن.

اگه صفحه رو Refresh کنیم، این تغییرات موقتی از بین می‌رن و صفحه دوباره از منابع اصلی بارگذاری می‌شه.

[صفحه رو Refresh کن و تغییرات رو نشون بده]

این ابزار برای توسعه‌دهنده‌ها خیلی مهمه؛ چون می‌تونن بدون اینکه هر بار فایل‌های پروژه رو تغییر بدن، ظاهر و رفتار صفحه رو بررسی کنن.

علاوه بر این، می‌تونن درخواست‌های شبکه، خطاهای JavaScript، وضعیت حافظه و اطلاعات دیگه‌ای رو هم مشاهده کنن.

فعلاً لازم نیست همه‌ی این تب‌ها رو بشناسید. فقط می‌خوام بدونید که مرورگر یه ابزار خیلی قدرتمند برای بررسی و عیب‌یابی صفحه‌های وب در اختیارمون قرار می‌ده.

اسلاید ۳۱ – دمو: کالبدشکافی یه صفحه‌ی وب ساده

خب، حالا می‌خوام یه مقدار از وب‌سایت‌های واقعی فاصله بگیریم و یه نمونه‌ی خیلی ساده‌تر رو بررسی کنیم.

من یه فایل آماده کردم که یه صفحه‌ی وب ساده رو نشون می‌ده.

[فایل anatomy.html رو باز کن و صفحه رو توی مرورگر نمایش بده]

این صفحه شاید از نظر ظاهری خیلی پیچیده نباشه، ولی برای آموزش ما خیلی مناسبه؛ چون می‌تونیم ببینیم HTML و CSS و JavaScript چطوری کنار هم کار می‌کنن.

حالا فایل رو توی ویرایشگر کد باز می‌کنم.

[فایل anatomy.html رو توی ویرایشگر باز کن؛ بخش HTML رو نشون بده]

توی قسمت HTML می‌تونیم ساختار صفحه رو ببینیم.

مثلاً عنوان، متن، تصویر، دکمه و بخش‌های مختلف صفحه، با استفاده از عناصر HTML تعریف شدن.

این عناصر مشخص می‌کنن که صفحه از چه قسمت‌هایی تشکیل شده.

حالا بریم سراغ قسمت CSS.

[بخش CSS رو نشون بده]

اینجا ویژگی‌های ظاهری صفحه تعریف شدن.

مثلاً رنگ‌ها، اندازه‌ها، فاصله‌ها، فونت‌ها و چیدمان عناصر.

حالا بخش JavaScript رو بررسی کنیم.

[بخش JavaScript رو نشون بده]

این قسمت رفتار صفحه رو کنترل می‌کنه.

مثلاً می‌تونیم مشخص کنیم که وقتی کاربر روی یه دکمه کلیک کرد، یه اتفاق خاصی بیفته.

خب، حالا بیاید یه تغییر خیلی کوچیک ایجاد کنیم.

[یه متن یا مقدار ساده رو توی HTML یا CSS تغییر بده، فایل رو ذخیره کن و صفحه رو Refresh کن]

دیدید؟

فقط با تغییر یه قسمت از کد، نتیجه رو بلافاصله توی مرورگر مشاهده کردیم.

این دقیقاً همون چرخه‌ایه که توسعه‌دهنده‌های وب دائماً باهاش کار می‌کنن:

کد می‌نویسن، نتیجه رو توی مرورگر می‌بینن، تغییرش می‌دن و دوباره نتیجه رو بررسی می‌کنن.

هدف من از نمایش این فایل این نیست که همین الان تمام این کدها رو یاد بگیرید.

می‌خوام وقتی اسم HTML و CSS و JavaScript رو می‌شنوید، بدونید هر کدوم چه نقشی دارن و چطوری با هم یه صفحه‌ی واقعی می‌سازن.

توی جلسه‌ی بعد قراره وارد همین موضوع بشیم و از نزدیک‌تر با HTML کار کنیم.

اسلاید ۳۲ – جمع‌بندی بلوک چهارم

خب، تا اینجا با چند تا مفهوم اساسی آشنا شدیم.

فهمیدیم مرورگر فقط یه پنجره برای دیدن وب نیست؛ بلکه یه نرم‌افزاره که منابع وب رو دریافت و پردازش می‌کنه.

دیدیم که طراحی وب با توسعه‌ی وب فرق داره، هرچند توی دنیای واقعی ممکنه این دو تا با هم هم‌پوشانی داشته باشن.

با سه تا فناوری اصلی صفحات وب هم آشنا شدیم:

HTML برای ساختار و معنای محتوا، CSS برای ظاهر و چیدمان و JavaScript برای منطق و تعامل.

همچنین تفاوت Frontend و Backend رو بررسی کردیم و دیدیم که یه وب‌سایت معمولاً از چند بخش مختلف تشکیل شده که با هم ارتباط دارن.

و در نهایت، با ابزار Developer Tools مرورگر آشنا شدیم و یه صفحه‌ی وب واقعی رو مورد بررسی قرار دادیم.

حالا احتمالاً تصویر روشن‌تری از این موضوع دارید که وقتی یه وب‌سایت رو باز می‌کنید، چه اجزایی پشت اون قرار دارن.

اما هنوز یه موضوع مهم باقی مونده.

امروزه ابزارهای هوش مصنوعی وارد دنیای برنامه‌نویسی شدن و حتی افرادی که هیچ تجربه‌ای توی برنامه‌نویسی ندارن، می‌تونن با کمک این ابزارها نمونه‌های اولیه‌ی وب‌سایت بسازن.

حالا سؤال اینجاست:

آیا این یعنی دیگه نیازی به یادگیری HTML و CSS و JavaScript نداریم؟

نه!

اتفاقاً این موضوع خودش یه بحث جالب داره که می‌خوایم توی بخش بعدی درباره‌ش صحبت کنیم.

بلوک پنجم: هوش مصنوعی و آینده طراحی وب

زمان: ۱۵ دقیقه | اسلایدهای ۳۳ تا ۳۶ | دقیقه ۷۵ تا ۹۰ کلاس

[دقیقه ۷۵ تا ۷۸ – برو اسلاید ۳۳]

هوش مصنوعی قراره جای برنامه‌نویس‌ها رو بگیره؟

خب، تا اینجا درباره ساختار صفحات وب و ابزارهایی که برای ساختنشون استفاده می‌کنیم صحبت کردیم.

اما یه اتفاق مهم توی چند سال اخیر افتاده که دنیای برنامه‌نویسی رو حسابی تغییر داده؛ اون هم ورود ابزارهای هوش مصنوعیه.

احتمالاً خودتون هم با ابزارهایی مثل Gemini، ChatGPT یا Claude کار کردین.

این ابزارها می‌تونن متن بنویسن، سؤال جواب بدن، کد تولید کنن و حتی توی پیدا کردن و رفع اشکال برنامه‌ها کمکمون کنن.

حالا یه سؤال مهم پیش میاد:

یعنی از این به بعد دیگه لازم نیست برنامه‌نویسی یاد بگیریم؟ کافیه به هوش مصنوعی بگیم چی می‌خوایم و خودش همه‌چیز رو برامون بسازه؟

خب، جواب کوتاه من اینه: آره، ولی نه دقیقاً!

ببینید، هوش مصنوعی می‌تونه سرعت انجام کارها رو خیلی بیشتر کنه. حتی می‌تونه برای کسی که تجربه برنامه‌نویسی نداره، یه نقطه شروع خیلی خوب باشه.

اما یه مسئله وجود داره: هوش مصنوعی همیشه درست نمی‌گه.

ممکنه کدی تولید کنه که در ظاهر درست کار می‌کنه، ولی یه جای کارش ایراد داشته باشه. ممکنه از روشی استفاده کنه که قدیمیه، یا اصلاً چیزی رو پیاده‌سازی کنه که با چیزی که ما می‌خواستیم فرق داشته باشه.

اینجاست که اهمیت دانش و شناخت ما مشخص می‌شه.

هرچی بیشتر بدونیم پشت صحنه چه خبره، بهتر می‌تونیم از هوش مصنوعی کمک بگیریم، نتیجه کارش رو بررسی کنیم و بفهمیم کجا اشتباه کرده.

این رو باید یادمون باشه که هوش مصنوعی در نهایت یه ابزاره و نه چیز دیگه.

[اینجا مثال نوشتن مثاله در زمینه مغز و اعصاب توسط یک مهندس نقشه بردار با استفاده از هوش مصنوعی رو بگو]

به نظرم بهترین نگاه به هوش مصنوعی این نیست که اون رو جایگزین کامل برنامه‌نویس بدونیم؛ بهتره به چشم یه دستیار قدرتمند بهش نگاه کنیم.

دستیاری که می‌تونه خیلی از کارها رو برامون سریع‌تر انجام بده، اما همچنان این ما هستیم که باید بدونیم چی می‌خوایم و نتیجه نهایی رو بررسی کنیم.

[دقیقه ۷۸ تا ۸۱ – برو اسلاید ۳۴]

هوش مصنوعی دقیقاً چه کمکی به ما می‌کنه؟

خب، حالا ببینیم این دستیار هوشمند دقیقاً چه کارهایی می‌تونه برامون انجام بده.

فرض کنید می‌خوایم یه وب‌سایت بسازیم، ولی هنوز نمی‌دونیم از کجا باید شروع کنیم.

می‌تونیم از هوش مصنوعی بخوایم چند ایده برای طراحی سایت بهمون بده، ساختار صفحات رو پیشنهاد کنه یا حتی یه نمونه اولیه از چیزی که توی ذهنمونه بسازه.

بعد می‌تونیم ازش بخوایم کد HTML و CSS اون صفحه رو تولید کنه.

مثلاً بگیم این دکمه رو تغییر بده، رنگ پس‌زمینه رو عوض کن، صفحه رو برای موبایل هم مناسب کن یا یه فرم بهش اضافه کن.

حتی اگه یه جایی به مشکل خوردیم، می‌تونیم کد رو بهش بدیم و بخوایم بررسی کنه که مشکل از کجاست.

یه قابلیت خیلی خوب دیگه هم اینه که می‌تونیم ازش بخوایم کدهایی رو که تولید کرده برامون توضیح بده.

یعنی به‌جای اینکه فقط یه تکه کد تحویل بگیریم، می‌تونیم بفهمیم هر قسمت چه کاری انجام می‌ده.

البته یه نکته رو فراموش نکنیم.

هوش مصنوعی ممکنه اشتباه کنه. ممکنه یه کد به ما بده که روی سیستم خودش درست به نظر می‌رسه، ولی وقتی اجراش می‌کنیم با خطا مواجه بشیم.

بنابراین نباید هر چیزی رو که تولید می‌کنه بدون بررسی استفاده کنیم.

یه نکته دیگه هم درباره اطلاعاتیه که در اختیار این ابزارها قرار می‌دیم. نباید اطلاعات محرمانه، رمز عبور، کلیدهای دسترسی یا اطلاعات شخصی و حساس رو بدون توجه به سیاست‌های حریم خصوصی سرویس، در اختیارشون قرار بدیم.

خب، حالا به‌جای اینکه فقط درباره این قابلیت‌ها حرف بزنیم، بیایید خودمون یه نمونه واقعی بسازیم.

[دقیقه ۸۱ تا ۸۷ – برو اسلاید ۳۵]

دمو: ساخت اولین صفحه وب با کمک هوش مصنوعی

خب، می‌خوام یه کار جالب انجام بدیم.

فرض کنید می‌خوایم یه کارت محصول برای یه فروشگاه اینترنتی طراحی کنیم.

این کارت قراره یه تصویر محصول داشته باشه، اسم محصول، یه توضیح کوتاه، قیمت و یه دکمه که با کلیک روش، محصول به سبد خرید اضافه بشه.

حالا فرض کنید من هنوز هیچ کدی ننوشتم.

می‌خوام از هوش مصنوعی کمک بگیرم تا این صفحه رو برام بسازه.

[مرورگر رو باز کن و ChatGPT یا ابزار هوش مصنوعی موردنظرت رو نمایش بده]

من اینجا به‌جای اینکه خیلی کلی بگم «یه سایت برام بساز»، سعی می‌کنم دقیق توضیح بدم چی می‌خوام.

چون هرچی درخواست ما روشن‌تر باشه، احتمال اینکه نتیجه به چیزی که توی ذهنمونه نزدیک‌تر بشه بیشتره.

مثلاً این درخواست رو براش می‌فرستم:

درخواستی که به هوش مصنوعی می‌دیم

یک کارت محصول فارسی و راست‌چین با HTML، CSS و JavaScript در یک فایل مستقل بساز.

کارت شامل تصویر محصول، نام، توضیح کوتاه، قیمت و دکمه افزودن به سبد خرید باشه.

طراحی مینیمال و حرفه‌ای باشه، در موبایل و دسکتاپ درست نمایش داده بشه و با کلیک روی دکمه، پیام تأیید نمایش داده بشه.

از کتابخانه خارجی استفاده نکن و کد رو خوانا و بخش‌بندی‌شده بنویس.

خب، درخواست رو ارسال می‌کنیم و منتظر می‌مونیم ببینیم چی تحویل می‌گیریم.

[اجازه بده هوش مصنوعی کد رو تولید کنه؛ خروجی رو نمایش بده]

ببینید! هوش مصنوعی برامون کد تولید کرده.

حالا یه نکته جالب اینجاست: ازش خواسته بودیم همه‌چیز رو توی یه فایل مستقل بنویسه. یعنی HTML و CSS و JavaScript همگی داخل همون یک فایل قرار دارن.

این کار برای شروع و اجرای یه نمونه ساده خیلی مناسبه.

حالا کد رو کپی می‌کنیم و داخل یه فایل با پسوند HTML ذخیره می‌کنیم.

[کد رو در ویرایشگر ذخیره کن؛ مثلاً با نام product.html]

حالا فایل رو با مرورگر باز می‌کنیم.

[فایل رو در مرورگر اجرا کن و نتیجه رو نمایش بده]

خب! این هم اولین صفحه‌ای که توی این کلاس با کمک هوش مصنوعی ساختیم.

حالا بیایید یه تغییر کوچیک هم توش ایجاد کنیم.

فرض کنید از ظاهر دکمه خوشمون نیومده. می‌تونیم دوباره به هوش مصنوعی بگیم ظاهر دکمه رو تغییر بده.

مثلاً ازش بخوایم دکمه گردتر باشه، رنگش متفاوت باشه و وقتی ماوس رو روش می‌بریم، یه تغییر ظاهری کوچیک داشته باشه.

[این درخواست اصلاحی رو به هوش مصنوعی بده]

«ظاهر دکمه افزودن به سبد خرید رو حرفه‌ای‌تر کن. گوشه‌های دکمه گردتر باشه، هنگام قرار گرفتن ماوس روی اون یه افکت ملایم داشته باشه و در نمایشگرهای موبایل هم اندازه مناسبی داشته باشه. بقیه بخش‌های صفحه رو تغییر نده.»

[خروجی جدید رو دریافت کن، در فایل جایگزین کن و نتیجه رو در مرورگر نشون بده]

می‌بینید؟ بدون اینکه خودمون مستقیماً کدنویسی کرده باشیم، تونستیم یه صفحه بسازیم و بعد هم ظاهرش رو تغییر بدیم.

البته این فقط یه نمونه خیلی ساده‌ست. ساخت یه وب‌سایت واقعی، مخصوصاً وقتی قراره اطلاعات کاربران رو نگهداری کنه یا امکانات پیچیده‌ای داشته باشه، به بررسی و کار بیشتری نیاز داره.

اما همین مثال نشون می‌ده که هوش مصنوعی چطور می‌تونه مسیر شروع کار رو برامون آسون‌تر کنه.

یه نکته مهم دیگه هم اینه که اگه از همین الان HTML و CSS و JavaScript رو بشناسیم، خیلی بهتر می‌تونیم با هوش مصنوعی کار کنیم. چون می‌فهمیم چی ازش بخوایم و می‌تونیم نتیجه‌ای رو که تحویل می‌ده بررسی کنیم.

[دقیقه ۸۷ تا ۹۰ – برو اسلاید ۳۶]

جمع‌بندی جلسه اول و نگاهی به جلسه بعد

خب دوستان، به آخر جلسه اول رسیدیم.

امروز سعی کردیم از یه تصویر خیلی بزرگ شروع کنیم و کم‌کم به اجزای کوچیک‌تر برسیم.

اول درباره خود اینترنت صحبت کردیم و دیدیم که اینترنت فقط یه فضای مجازی و نامرئی نیست؛ بلکه یه زیرساخت عظیم از شبکه‌ها، کابل‌ها، تجهیزات و ارتباطات مختلفه.

بعد تفاوت اینترنت و وب رو بررسی کردیم و دیدیم که وب فقط یکی از سرویس‌هاییه که روی اینترنت کار می‌کنه.

بعدش سراغ اتفاقاتی رفتیم که وقتی یه آدرس رو توی مرورگر وارد می‌کنیم، پشت صحنه رخ می‌دن. با دامنه، DNS ،IP، سرور، کلاینت و مفهوم درخواست و پاسخ آشنا شدیم.

در ادامه هم درباره خود صفحات وب صحبت کردیم و فهمیدیم که سه فناوری اصلی HTML، CSS و JavaScript چطور کنار هم قرار می‌گیرن و یه صفحه وب رو می‌سازن.

با مفهوم Frontend و Backend آشنا شدیم و دیدیم که مرورگر فقط یه ابزار ساده برای دیدن سایت‌ها نیست؛ بلکه کلی ابزار برای بررسی و توسعه صفحات وب در اختیارمون قرار می‌ده.

و در آخر هم یه نمونه ساده رو با کمک هوش مصنوعی ساختیم.

اما از جلسه بعد قراره کم‌کم از حالت تماشاگر خارج بشیم و خودمون دست به کار بشیم.

می‌خوایم یاد بگیریم چطور یه صفحه وب رو از صفر بسازیم.

اول با HTML شروع می‌کنیم؛ زبانی که اسکلت و ساختار صفحات وب رو تشکیل می‌ده.

قرار نیست از همون ابتدا وارد مباحث پیچیده بشیم. قدم‌به‌قدم جلو می‌ریم، مثال می‌زنیم، کد می‌نویسیم و نتیجه رو توی مرورگر می‌بینیم.

هدف نهایی اینه که تا پایان دوره بتونیم یه وب‌سایت واقعی بسازیم و منتشرش کنیم.

پس فعلاً همین‌قدر از جلسه اول کافیه.

ممنون که همراه بودین. امیدوارم از این جلسه لذت برده باشین.

تا جلسه بعد، مراقب خودتون باشین! 🌱